Fix inappropriate font choices for "declaration"

Document number:
P3924R1

Date:
2026-03-23

Audience:
CWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:
Jan Schultke <janschultke@gmail.com>

GitHub Issue:
wg21.link/P3924/github

Source:
github.com/eisenwave/cpp-proposals/blob/master/src/fix-declaration-font.cow

---

Contents

1. Revision history

1.1. Changes since R0

- In § [dcl], added **the** after "or" in first two changes

§ 2. Introduction

The C++ standard has two similar terms:

- the regular-font "declaration" defined in [basic.pre]
- the grammatical *declaration* defined in [dcl]

> EXAMPLE: There exist declarations that are not *declaration*s, such as the *elaborated-type-specifier* `struct S`.

There are instances where these have been used incorrectly. NB comment US 11-400 requests:

> Perform a thorough review of each usage of the term "declaration" to confirm that it is rendered in the correct style.

Such a review has been performed. The vast majority of occurrences are "declaration", not *declaration*. Since "declaration" is somewhat of a superset, the term can almost always be used without grammatical font. Using grammatical font is more likely to result in a mistake, so it should be done with great caution and confidence.

In some cases, the surrounding wording is adjusted to fit the existing use of *declaration*.

§ 3. Wording

> DRAFTING NOTE: Some mistakes in [N5014] have already been fixed, such as the misuse in [basic.pre] pointed out by US 12-026.

§ [basic]

Change [basic.def] paragraph 2 as follows:

> Each entity declared by a declaration is also *defined* by that declaration unless:
>
> - [...]
> - it is an ~~explicit specialization~~ ***explicit-specialization*** ([temp.expl.spec]) whose *declaration* is not a definition.

Change [basic.scope.scope] note 1 as follows:

> [*Note*: Special cases include that:

> - [...]
> - The ~~declaration in~~ ***declaration of*** a *template-declaration* inhabits the same scope as the *template-declaration*.
> - [...]
>
> — *end note*]

Do not change [basic.link] paragraph 1:

> A program consists of one or more translation units ([lex.separate]) linked together. A translation unit consists of a sequence of declarations.
>
> *translation-unit*:
> *[...]*

DRAFTING NOTE: The "introductory" wording at the start of subclauses that introduce various syntactical constructs could use grammatical font in many cases. However, this is a general issue, not specific to "declaration", and should be addressed holistically.

[stmt]

Do not change [stmt.pre] paragraph 7:

> If a *condition* can be syntactically resolved as either an expression or a declaration, it is interpreted as the latter.

DRAFTING NOTE: This paragraph is updated separately, in [CWG3132].

Change [stmt.block] paragraph 2 as follows:

> [*Note*: A compound statement defines a block scope ([block.scope]). ~~A declaration is a *statement* ([stmt.dcl]).~~ — *end note*]

DRAFTING NOTE: Not every declaration is a *statement*, only a *declaration-statement* is. The second sentence is also unnecessary in general because the surrounding wording and the grammar are already clear.

Change [stmt.expand] paragraph 5, bullet 3 as follows:

> Otherwise, S is a destructuring expansion statement and S is equivalent to:
>
> {
> *init-statement*
> `constexpr`$_{opt}$ `auto&& [`u_0`, `u_1`, …, `u_{N-1}`] =` *expansion-initializer*`;`
> S_0
> ⋮

S_{N-1}
}

where N is the structured binding size of the type of the *expansion-initializer* and S_i is

{
for-range-declaration = u_i ;
compound-statement
}

The keyword `constexpr` is present in the ~~declaration~~ ***structured-binding-declaration*** of $u_0, u_1, \ldots, u_{N-1}$ if and only if `constexpr` is one of the *decl-specifier*s of the *decl-specifier-seq* of the *for-range-declaration*.

§ [dcl]

Change [dcl.stc] paragraph 1 as follows:

[...] If a *storage-class-specifier* appears in a *decl-specifier-seq*, there can be no `typedef` specifier in the same *decl-specifier-seq* and the *init-declarator-list* **of the *simple-declaration*** or **the** *member-declarator-list* of the ~~declaration~~ ***member-declaration*** shall not be empty (except for an anonymous union declared in a namespace scope ([class.union.anon])). [...]

Change [dcl.type.cv] paragraph 1 as follows:

[...] If a *cv-qualifier* appears in a *decl-specifier-seq*, the *init-declarator-list* **of the *simple-declaration*** or **the** *member-declarator-list* of the ~~declaration~~ ***member-declaration*** shall not be empty. [...]

Do not change [dcl.type.elab] paragraph 2 as follows:

> If an *elaborated-type-specifier* is the sole constituent of a declaration, the declaration is ill-formed unless it is an explicit specialization ([temp.expl.spec]), a partial specialization ([temp.spec.partial]), an explicit instantiation ([temp.explicit]), or it has one of the following forms:
>
> [...]

DRAFTING NOTE: If the wording was restricted to *declaration*s, this rule may not apply because grammatically, a *declaration* cannot solely consist of a *elaborated-type-specifier*.

Change [dcl.decl.general] paragraph 3 as follows:

Each *init-declarator* **of a *simple-declaration*** or *member-declarator* **of a *member-declaration*** ~~in a declaration~~ is analyzed separately as if it were in a ~~declaration~~ ***simple-declaration* or *member-declaration*** by itself.

Do not change [dcl.ambig.res] paragraph 1:

> [...] However, a construct that can syntactically be a *declaration* whose outermost *declarator* would match the grammar of a *declarator* with a *trailing-return-type* is a declaration only if it starts with `auto`.

DRAFTING NOTE: The above also applies to things like function parameters, which are not *declaration*s, but may syntactically match a *declaration*.

Change [dcl.meaning.general] paragraph 4 as follows:

> A `static`, `thread_local`, `extern`, `mutable`, `friend`, `inline`, `virtual`, `constexpr`, `consteval`, `constinit`, or `typedef` specifier or an *explicit-specifier* applies directly to each *declarator-id* in a ~~declaration~~ ***simple-declaration* or *member-declaration***; the type specified for each *declarator-id* depends on both the *decl-specifier-seq* and its *declarator*.

§ [class.dtor]

Do not change [class.dtor] paragraph 1:

> A declaration whose *declarator-id* has an *unqualified-id* that begins with a `~` declares a *prospective destructor*; its *declarator* shall be a function declarator ([dcl.fct]) of the form [...]

DRAFTING NOTE: A *declarator-id* does not directly belong to a *declaration*. This wording seems to make use of the fact that separate *declarator*s in a single *declaration* are separate declarations.

Do not change [class.conv.fct] paragraph 1:

> A declaration whose *declarator-id* has an *unqualified-id* that is a *conversion-function-id* declares a *conversion function*; its *declarator* shall be a function declarator ([dcl.fct]) of the form [...]

§ [over.literal]

Do not change [over.literal] paragraph 2:

> A declaration whose *declarator-id* is a *literal-operator-id* shall declare a function or function template [...]

§ 4. References

[N5014] *Thomas Köppe.* Working Draft, Programming Languages — C++ (2025-08-05)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf

[CWG3132] *CWG.* Unclear disambiguation rule for condition (2025-11-21)
https://cplusplus.github.io/CWG/issues/3132.html